

CUCEI



UdeG

Algoritmo: Colonia de Abejas Artificiales

02 de Octubre de 2026

Algoritmos Metaheurísticos

Maestra: Paredes López Ángel Ignacio

Alumno: Mariscal Rodríguez Omar Jesús (220858478)

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
ALGORITMOS METAHEURÍSTICOS
Prof, Ángel Ignacio Ángel Paredes

Indicaciones Generales:	3
Marco Teórico y Diseño del Algoritmo:	5
Sintonización de Parámetros	8
Análisis Estadístico y Curva de Convergencia Promedio	10
Distribución Espacial de la Población (Visualización 2D)	12
Discusión Analítica	13
Anexo del Código	14
Conclusión	57

INDICACIONES GENERALES:

Implementar el **Algoritmo de Colonia de Abejas Artificiales (Artificial Bee Colony - ABC)**. A diferencia de los algoritmos genéticos, ABC modela el comportamiento de forrajeo de las abejas dividiendo la población en tres fases operativas: abejas empleadas, abejas observadores y exploradoras.

El objetivo de esta actividad es evaluar la capacidad de exploración y explotación de ABC al enfrentarse a una superficie no convexa y con una enorme cantidad de mínimos locales: la función **Eggholder**.

Funciones a Optimizar:

1. Función de Ackley (2D): *(Para comparación directa con la Actividad 2)*

$$f(x, y) = - (y + 47) \sin(\sqrt{|y + \frac{x}{2} + 47|}) - x \sin(\sqrt{|x - (y + 47)|}), \quad x, y \in [-512, 512]$$

(Nota: El óptimo global conocido se encuentra en $f(512, 404.2319) \approx -959.6407$)

Metodología y Configuración de Parámetros: Deberá realizar un proceso de sintonización empírica para determinar el tamaño óptimo de la colonia (SN), el número máximo de ciclos/iteraciones y, de manera crucial, el parámetro límite de abandono de fuente de alimento (*limit*), el cual suele definirse en función de la dimensión y la población (ej. $limit = SN \times D$)

Dado el carácter estocástico del algoritmo y la complejidad del paisaje de búsqueda, todo reporte de resultados debe basarse en la media estadística de al menos **30 ejecuciones independientes** con semillas aleatorias distintas.

Entregables del Reporte (PDF):

- **Marco Teórico y Diseño del Algoritmo:** Explique matemáticamente la lógica de actualización de soluciones mediante la búsqueda vecinal $v_{ij} = x_{ij} + \phi_{ij} (x_{ij} - x_{kj})$ el mecanismo de selección probabilística por ruleta en las abejas observadoras y la función del parámetro *limit* para activar las abejas exploradoras.
- **Sintonización de Parámetros:** Presenta una tabla o gráfica justificando la elección de la cantidad de abejas (SN) y el valor de *limit*. Explique qué sucede si *limit* es demasiado pequeño o demasiado grande.
- **Análisis Estadístico y Curva de Convergencia Promedio:** Reporte en una tabla el Mejor Fitness, Peor Fitness, Media y Desviación Estándar de las 30 ejecuciones. Incluya la gráfica de convergencia promedio (fitness del mejor individuo vs. iteraciones)
- **Distribución Espacial de la Población (Visualización 2D):** Presente tres gráficas de dispersión o contorno sobre el plano $[-512, 512]^2$ mostrando la

ubicación geográfica de las fuentes de alimento (población) en tres momentos: Generación Inicial, Generación Intermedia y Generación Final.

- **Discusión Analítica (Pregunta Clave):** La función Eggholder es famosa por sus ‘trampas’ de mínimos locales ¿De qué manera la fase de abejas exploradoras previene que el enjambre sufra de estancamiento prematuro? ¿Qué diferencia existe entre la mutación de un algoritmo genético y el abandono de solución en ABC?
- **Anexo de Código:** Adjuntar el código fuente debidamente comentado.
- **Conclusiones:** Describir los hallazgos y conclusiones finales de la práctica.

Evaluación Presencial y Modificación de Código: El 40% de la calificación de esta actividad se acreditará mediante la modificación presencial del código en clase, la cual se realizará exclusivamente el día 5 de octubre (una sesión antes del límite de entrega del reporte).

MARCO TEÓRICO Y DISEÑO DEL ALGORITMO:

Arquitectura del Código:

Como una breve mención, el código sigue los mismos estándares que las actividades anteriores con el **Patrón de Estrategia** para facilitar los cambios en los procesos específicos del algoritmo (como la selección de la fuente vecina o cantidad de fuentes de comida que se pueden abandonar por ciclo, etc.) con un conjunto de estas estrategias codificadas que cumplen con un **Protocolo o Contrato** para garantizar la estabilidad del código ante adiciones.

Se divide en dos archivos: **abc_colony.py** que esencialmente es el motor de configuraciones y el corazón del algoritmo, (el equivalente al `ga_continuo.py` de la actividad anterior) y el **main.py** que hace de orquestador y visualización de resultados.

Más allá de eso, el código reutiliza los modos de ejecución default y tuning, este último para facilitar la prueba del mismo algoritmo bajo las mismas condiciones con diferentes combinaciones de hiper parámetros para encontrar la combinación ideal y evaluar sus resultados con diferentes atributos.

Funcionamiento Matemático de las Operaciones ABC:

En esta sección, se explica en un lenguaje matemático como es que funcionan las operaciones que internamente calcula el código:

- Lógica de actualización por la búsqueda vecinal:

La búsqueda de una mejor fuente de alimento en las proximidades de la fuente actual en referencia con una fuente vecina es una acción que ejecutan tanto las abejas empleadas cuando ‘llegan’ a la fuente de alimento y las abejas observadoras cuando, por un método de selección llegan a una fuente de alimento; se desplazan de su fuente en una dirección a la de referencia. El objetivo es precisamente, encontrar una mejor fuente que la que actual. El proceso cuenta con cuatro fases:

- Selección de la fuente vecina:

Como el cálculo de desplazamiento se hace con la fuente actual y una fuente de referencia vecina, el primer paso es seleccionar entre las $SN - 1$ fuentes totales de alimento (restando la propia), una para desplazarnos hacia ella. Esta selección, en el código se hace de una de dos formas programadas; Una **selección aleatoria o la más cercana geográficamente vía distancia euclidiana**. En la primera, el proceso es simple, solo se toma un índice aleatorio distinto de k (donde k es la fuente actual) y el seleccionado será el elegido para hacer el desplazamiento vecinal.

Por otro lado, la segunda estrategia consiste en seleccionar como vecino al más cercano geográficamente por la distancia euclidiana:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Esto trae beneficios distintos: mientras que la selección aleatoria favorece la aleatoriedad y una mayor riqueza en el espacio de búsqueda total; la distancia euclidiana converge más rápido a una solución próxima a la fuente.

- Cálculo de la vecindad:

El siguiente paso, una vez elegida la referencia es que la abeja explora la nueva fuente de comida, se desplaza hasta ella bajo la siguiente fórmula:

$$v_{ij} = x_{ij} + \phi_{ij} (x_{ij} - x_{kj})$$

Donde ϕ es un número aleatorio entre $[-1,1]$. Si notamos, este cálculo se va a crecer en función de la distancia entre las fuentes de alimento, si una fuente A está muy alejada de la fuente B, el desplazamiento será más grande.

- Evaluación de la fuente:

La abeja, una vez llegado a esa fuente de comida, va a evaluar la cantidad de néctar que hay en la nueva fuente, (en nuestro caso es el fitness), en dado caso que esta nueva fuente tenga un mejor fitness, la abeja olvida por completo la fuente de alimento anterior y se queda en esta nueva, reiniciando un contador de estancamiento a 0 nuevamente.

En dado caso de que la nueva fuente sea peor que la anterior, la abeja vuelve a la fuente original y aumenta el contador de estancamiento de dicha fuente en +1.

Este contador de estancamiento se relaciona con la sección de las abejas exploradoras que se detalla también aquí:

- **Mecanismo de selección probabilística por ruleta de las abejas observadoras:**

Una vez las abejas empleadas vuelven a la colmena habiendo hecho su trabajo de la búsqueda vecinal y con resultados de la cantidad de néctar que hay en cada fuente de alimento, las abejas exploradoras recopilan esta información, cada una de ellas (típicamente hay SN cantidades de abejas observadoras, igual que abejas empleadas) va a seleccionar una fuente de alimento por *ruleta*.

La ruleta funciona igual que en los algoritmos anteriores, se tiene un 100% de probabilidad que será repartido entre cada una de las SN fuentes de alimento de manera proporcional a su fitness; como queremos minimizar, las fuentes de alimento con menor fitness tendrán, proporcionalmente un mayor porcentaje en esta ruleta, y los fitness más altos una menor. Al final, las probabilidades deben representar fielmente el rendimiento de los individuos, beneficiando a los mejores pero sin desaparecer a los peores.

Entonces, cada abeja 'rueda' esta ruleta y la fuente que toque irá a ella, entonces, hará el mismo proceso que una abeja empleada, aplicando la lógica de actualización mediante la búsqueda vecinal. Cabe destacar que este proceso también puede aumentar el contador de estancamiento de las fuentes de alimento. La diferencia aquí es que en las empleadas, la relación abeja-fuente es 1:1, por otro lado, la relación con las

observadoras puede varias. Las mejores soluciones tenderán a tener a su alrededor más abejas observadoras en búsqueda de mejorarla aún más y las peores tenderán a tener pocas o ninguna abeja observadora que quiera explorarlo.

El punto aquí es precisamente premiar a las mejores soluciones con más exploraciones.

- Abejas exploradoras y la función del parámetro *limit*:

Cada una de las fuentes de alimento tiene asignado un contador de estancamiento inicializado en 0, cada vez que un intento de mejora por parte de una abeja falla, el contador incrementa en +1, si la fuente fue reemplazada por uno mejor, el contador se reinicia a 0.

En dado caso de que una fuente de alimento sobrepase en su contador de estancamiento el parámetro *limit*, entrarán en acción las abejas exploradoras. El proceso consta en abandonar la fuente de alimento o solución estancada completamente y seguir el proceso de dicha fuente en un punto completamente aleatorio en todo el espacio de búsqueda. Es la herramienta que el algoritmo tiene para evitar el estancamiento y dar con mejores soluciones en funciones con mínimos locales engañosos.

Ahora, el código implementa dos opciones configurables: La primera es limitar el abandono de fuentes de alimento a 1 por iteración, donde se abandona el peor, y el otro es más agresivo dejando todas las fuentes a la vez que hayan sobrepasado el *limit* en su estancamiento.

Una vez se pasa por las tres fases de las abejas: Empleadas, observadores y exploradoras (si aplica), se considera que el algoritmo ha dado una iteración.

SINTONIZACIÓN DE PARÁMETROS

Para esta parte de encontrar la mejor configuración de parámetros se utilizó el modo **tuning** del código, que como ya se explicó anteriormente, se basa en pocas corridas del algoritmo (10 específicamente) con cierta combinación de hiper parámetros, registrando los resultados de cada combinación y así, automatizar la colección de datos, la tabla que se adjunta fueron los resultados obtenidos:

SN	limit	max_cycles	media_mejor	desv_std	tasa_exito	ciclos_mediana
20	20	200	-955.599950	3.846571	0.3000	104.70
20	20	400	-955.599950	3.846571	0.3000	104.70
20	40	200	-953.851216	13.542931	0.7000	125.60
20	40	400	-953.851216	13.542931	0.7000	125.60
20	80	200	-939.141931	25.856592	0.4000	123.70
20	80	400	-942.161796	23.871750	0.4000	143.70
40	40	200	-958.834106	1.730516	0.8000	111.20
40	40	400	-958.834106	1.730516	0.8000	111.20
40	80	200	-958.353099	2.341219	0.7000	113.10
40	80	400	-958.353099	2.341219	0.7000	113.10
40	160	200	-959.461623	0.360054	0.9000	151.30
40	160	400	-959.583443	0.146593	1.0000	179.50
80	80	200	-959.402891	0.707296	0.9000	136.80
80	80	400	-959.402891	0.707296	0.9000	139.80
80	160	200	-959.636563	0.012131	1.0000	167.30
80	160	400	-959.636563	0.012131	1.0000	190.90
80	320	200	-959.636551	0.012127	1.0000	175.10
80	320	400	-959.636555	0.012128	1.0000	215.00

A primera vista en la tabla, no es de sorprenderse que mientras vamos subiendo la cantidad de fuentes de comida (y con ello, la cantidad de abejas totales en la colmena) nos aproximamos bastante más cerca del mínimo global de la función Eggholder, llegando a una muy buena precisión con una desviación estándar bastante chica y tasas de éxito del 100% esto claro porque aumentamos el muestreo de la exploración del espacio de búsqueda significativamente. Sin embargo, esto también aumenta la carga computacional requerida para completar el algoritmo.

Por dar un ejemplo, la primera ejecución con únicamente 20 fuentes de alimento tardó solo 1.0s en completarse, mientras que la última con 80 fuentes de alimento tardó aproximadamente 6.5s. Se sextuplicó el tiempo y esto puede ser exponencial, aquí entra un factor importante sobre *cuál es nuestro margen de error aceptable*. No es lo mismo que dependamos de obtener una cifra significativa después del punto a 4, que cada vez se hace más complicado y costoso. Nosotros vamos a partir del supuesto que el costo no

es un gran problema, así que podemos darnos el lujo de elegir una cantidad de fuentes de alimento elevada.

Entre 40 y 80 como opciones de fuentes de alimento son las que consiguen una precisión de máximo un punto de error, así que entre estas está nuestra elección. El resultado con el que vamos a elegir una sobre otra será la desviación estándar, con 40 esta está entre 2.x y 0.x, mientras que en los 80 está típicamente en el rango 0.0x, una diferencia considerable, esto nos dice que con 80 es más factible que el algoritmo es sólido y que no dependemos tanto de la suerte para encontrar buenos resultados sino que la mayoría de las veces llega a uno bueno, esto sin mencionar que la tasa de éxito se mantuvo prácticamente en 100% en casi todas las combinaciones de 80 SN no como en los 40 SN.

El número máximo de ciclos es, seguramente el apartado con menos impacto, por lo general, el algoritmo para por condición de paro débil en la mayoría de los casos, pero, y siguiendo el supuesto que en nuestro caso, el costo computacional no es un gran problema dada la función, (la selección de hiper parámetros puede variar de problema en problema aunque sea el mismo algoritmo), elegiremos 400 para propiciar más paros débiles que el fuerte.

El parámetro limit es un mecanismo de control crucial en el algoritmo ABC, ya que determina el momento en el que se debe abandonar una fuente de alimento agotada (mínimo local) para activar la fase de las abejas exploradoras e inyectar diversidad. Habitualmente, se recomienda definir este límite en función de la dimensión del problema (D) y el tamaño de la población (SN) mediante la relación $\text{limit} = \text{SN} \times D$. Los resultados de la tabla demuestran de forma empírica la importancia de este balance:

Si el límite es muy bajo, las abejas abandonan las fuentes de alimento de forma prematura antes de haberlas explotado exhaustivamente. Esto convierte la ejecución en una búsqueda con exceso de exploración, lo que dificulta la convergencia fina. En la tabla se evidencia que para SN=20, al utilizar un limit restrictivo de 20, la tasa de éxito fue de apenas un 30%, con un estancamiento del mejor fitness medio en -955.5999

Si el límite es excesivamente alto, las abejas desperdician una gran cantidad de ciclos intentando mejorar un mínimo local que ya no tiene potencial (exceso de explotación). Esto retrasa dramáticamente la aparición de las abejas exploradoras, provocando un estancamiento prematuro en el enjambre. Los datos muestran que, para la misma configuración de SN=20, al elevar el limit a 80 el rendimiento general cayó a un 40% de éxito, y la media empeoró hasta -939.1419 o -942.1618, dependiendo del número de ciclos. Pensando en ello, el *limit* que elegiremos será 160 (que concuerda con la recomendación de $\text{limit} = \text{SN} \times D$)

Entonces, en resumen, los hiper parámetros serán:

- **SN: 80**
- **limit: 160**
- **max_cyc: 400**

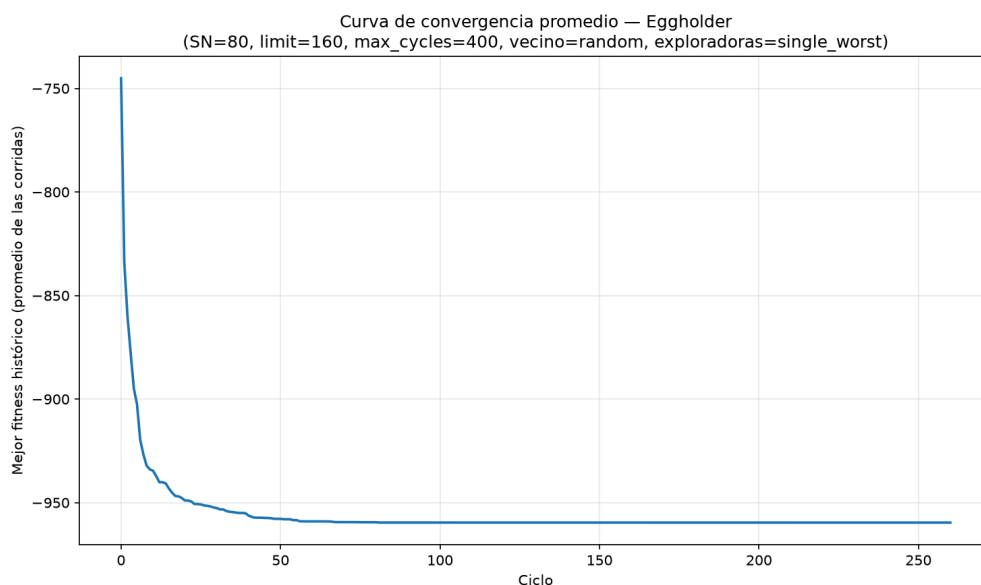
ANÁLISIS ESTADÍSTICO Y CURVA DE CONVERGENCIA PROMEDIO

En esta sección se ejecutará el modo **final** o **default** del programa con los hiper parámetros seleccionados de la opción pasada y las estrategias comunes del ABC, en 30 ejecuciones estocásticas para evaluar su comportamiento en la función Eggholder, a continuación los resultados:

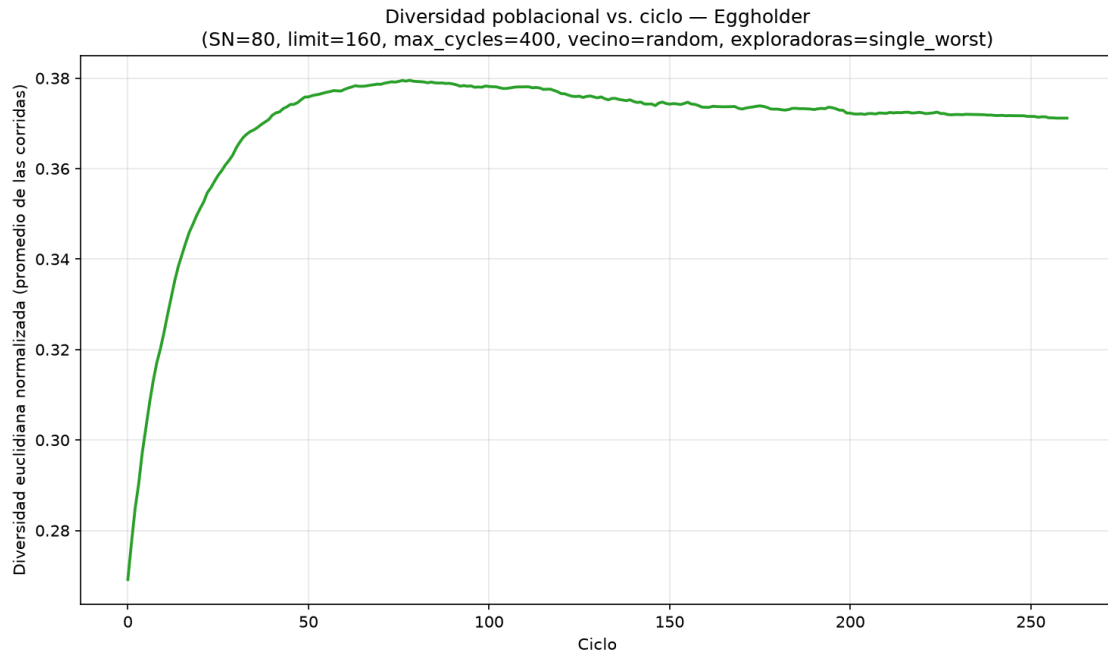
problema	mejor	peor	media	desv_std	tasa_exito	ciclos_media
eggholder	-959.640663	-959.444699	-959.629315	0.037193	1.0000	168.10

Los resultados de las 30 corridas son tan buenos como se podría esperar dada la selección de hiper parámetros que hicimos y no limitando tanto el costo computacional; obtuvimos un 100% de tasa de éxito, en todas nuestras ejecuciones se llegó a un aproximado notable con el mínimo global conocido, y a pesar de fijar como máximo 400 ciclos, el promedio de paro fue en 168.10, es decir, normalmente se paró por un criterio de paro débil que considerando que nuestros resultados se aproximan bastante al valor real, buena señal.

Además, nuestra desviación estándar es muy baja, esto también se refleja entre la diferencia del mejor, peor y la media de los resultados, todos tienen cantidades muy cercanas al valor por desfasar de punto decimal, y la media, discrepancias solo a partir de la segunda cifra significativa después del punto decimal.



En la gráfica de convergencia vemos una disminución bastante rápida del fitness, que a partir de la generación 50 se hicieron microajustes, esto quiere decir que, otra manera que tenemos de limitar el gasto computacional en función de que tan precisa queremos la solución, con este algoritmo y estas configuraciones podríamos acotar bastante el número de iteraciones, considerando que es apenas un 20% de los ciclos donde se hacen correcciones significativas.

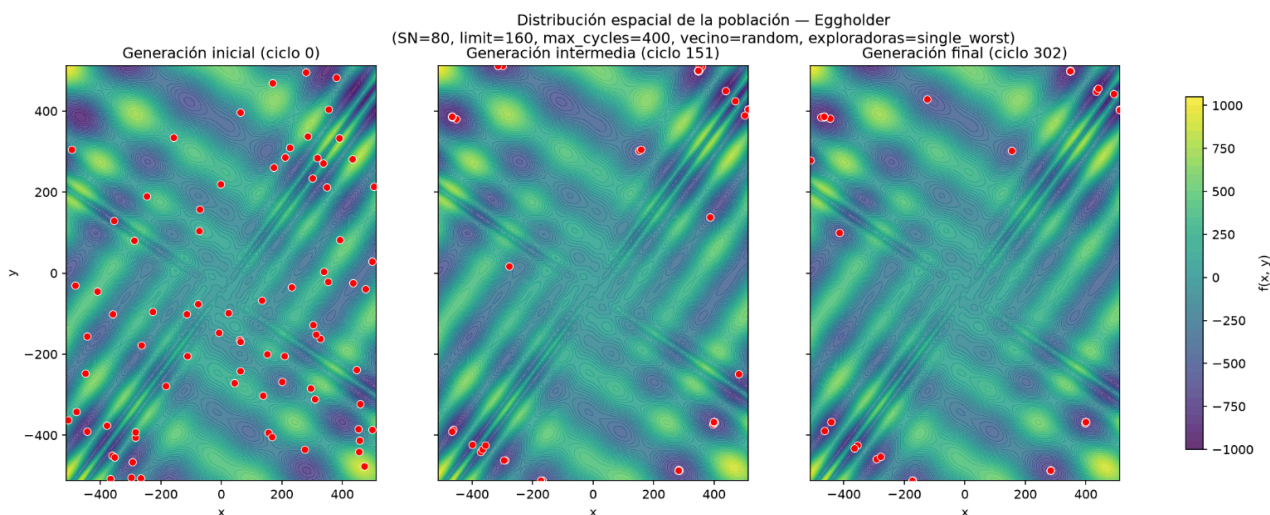


También se adjunta una gráfica de como la diversidad poblacional de forma euclidiana y normalizada por cada ciclo.

Como se ve en el código, este tema de la diversidad también es un criterio de paro débil sobre el cual el código pretende tener siempre diversidad poblacional y no rodear las mismas soluciones.

DISTRIBUCIÓN ESPACIAL DE LA POBLACIÓN (VISUALIZACIÓN 2D)

Para analizar la dinámica de búsqueda del ABC, se mapeó la ubicación espacial de la población (las fuentes de alimento) sobre las curvas de nivel de la función Eggholder en tres etapas críticas de su ejecución.



Las gráficas de contorno utilizan un gradiente de color donde las áreas púrpuras oscuras representan las regiones de menor valor de aptitud (los mínimos profundos, aproximándose a -1000) y las áreas amarillas representan los picos de mayor costo. La simulación se realizó utilizando los parámetros óptimos anteriormente determinados

- La primera gráfica ilustra la inicialización estocástica del modelo matemático. Las 80 fuentes de alimento se encuentran dispersas de manera aleatoria y uniforme a lo largo del dominio acotado. En esta etapa, la diversidad espacial de la colonia es máxima, lo que garantiza una cobertura global inicial del espacio de búsqueda antes de aplicar presión de selección.
- Durante la fase transicional, se evidencia empíricamente la capacidad de explotación local inducida por las abejas empleadas y la selección probabilística por ruleta de las abejas observadoras. La población ha abandonado las zonas topográficas desfavorables (áreas amarillas) y ha migrado hacia las cuencas de atracción locales. Debido a la naturaleza no convexa de la función Eggholder, el enjambre se fragmenta y explora múltiples vallas subóptimas de manera simultánea.
- La visualización final muestra la convergencia del algoritmo hacia las regiones de mayor profundidad topográfica, agrupando densamente a la población en las vecindades de los mejores mínimos identificados. Es crucial notar la presencia de individuos aislados distribuidos de forma dispersa fuera de los clústeres principales. Estos puntos representan la activación de las abejas exploradoras. Al sobrepasar el límite de estancamiento, las fuentes agotadas fueron descartadas y inicializadas en posiciones aleatorias, probando visualmente que el mecanismo de inyección de diversidad opera de buena manera.

DISCUSIÓN ANALÍTICA

¿De qué manera las abejas exploradoras previenen que el enjambre sufra de estancamiento prematuro?

Cuando el enjambre converge hacia uno de los valles subóptimo, las abejas empleadas y observadoras agotan rápidamente su capacidad de mejora local mediante la ecuación de búsqueda vecinal. En este punto, el estancamiento es inminente.

La fase de las exploradoras previene este escenario al superarse la tolerancia del límite sin mejoras en la aptitud de la fuente, el algoritmo asume que el mínimo local ha sido explorado por completo. La fuente es descartada y la abeja se transforma en exploradoras, lo que obliga al sistema a inyectar una nueva solución generada de manera aleatoria en cualquier coordenada del espacio de búsqueda.

Esta inyección drástica de diversidad macroespacial permite al enjambre saltar fuera de los mínimos locales y descubrir regiones del paisaje que de otro modo quedarían ignorados.

Diferencias entre la mutación (Algoritmo Genético) y el abandono de solución (ABC)

Aunque ambos mecanismos tienen el objetivo de mantener la diversidad y evitar la convergencia prematura, operan bajo paradigmas matemáticos y estructurales distintos.

La mutación es una perturbación local, esto es, el operador de modificación que aplica una alterada probabilística sobre la estructura de solución existente. Ya sea alterando valores en un vector de coma flotante o cambiando bits, la mutación retiene gran parte del material genético o la memoria posicional del individuo original.

Su función es explorar la vecindad inmediata de la solución para evitar que la población se vuelva completamente homogénea.

Por otro lado, el abandono de la solución en ABC es un reemplazo global y agresivo\

No es una perturbación sino un descarte absoluto. Cuando una solución supera el umbral del parámetro limit se destruye por completo toda la información posicional e histórica de ese individuo. La nueva fuente de alimento descubierta por la abeja exploradora no guarda ninguna relación matemática ni dependencia especial con la solución abandonada; se inicializa desde cero.

Mientras que la mutación genética busca realizar pequeños ajustes, el abandono de ABC es un mecanismo de reinicialización global agresiva diseñado para realizar saltos de exploración de larga distancia en la topografía del problema.

ANEXO DEL CÓDIGO

abc_colony.py

"""

Autor: Mariscal Rodríguez Omar Jesús

Materia: Algoritmos Metaheurísticos

Profesor: Paredes López Ángel Ignacio

Actividad 4 - Algoritmo de Colonia de Abejas Artificiales

Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

Motor del Algoritmo de Colonia de Abejas Artificiales (Artificial
Bee Colony,

ABC) para minimización de funciones con dominio restringido tipo
caja.

Arquitectura (mismo espíritu que ga_binario.py / ga_continuo.py de
las

actividades previas):

- GAConfig pasa a ser aquí ABCConfig: dataclass que centraliza
la parametrización.

- Representación real (fuentes de alimento = vectores de
floats), sin

- codificación adicional.

- Operadores intercambiables vía Protocol + registry dado el
patrón de diseño de estrategia:

- NeighborSelectionOperator: cómo se elige el vecino k en la
$$\text{búsqueda vecinal } v_{ij} = x_{ij} + \phi_{ij} \cdot (x_{ij} - x_{kj})$$
 - "random": ABC clásico: k aleatorio $\neq i$. Este es el visto en clase
 - "nearest_euclidean": variante encontrada: k = vecino más cercano en
$$\text{distancia euclidiana (mayor explotación local, menor exploración que random)}.$$
- ScoutStrategy: qué fuentes se abandonan/reinician cuando superan
 'limit' intentos sin mejorar.
 - "single_worst": ABC clásico: solo la fuente más agotada
$$\text{que supere 'limit', una por ciclo.}$$
 - "all_exceeding": variante: Todas las fuentes que superen
$$\text{'limit'}$$
 se reinician en el mismo ciclo.
- StoppingCriterion: paro fuerte (máx. ciclos), paro débil (estancamiento del mejor histórico) y un criterio opcional de colapso de diversidad
$$\text{(basado en distancia euclidiana media al centro), desactivado por defecto.}$$
- Se registra en cada ciclo, además de la convergencia, la diversidad poblacional (distancia euclidiana media de las fuentes al centroide,

normalizada por la diagonal del dominio) – información extra útil

para ilustrar cómo las exploradoras reinyectan diversidad.

Las 3 fases del ciclo ABC son cada una de los tipos de abejas vistas:

1. Abejas Empleadas: cada una de las SN fuentes intenta una búsqueda

vecinal (greedy: se queda con el mejor entre la fuente actual y la

candidata).

2. Abejas Observadores: se seleccionan SN fuentes (con reemplazo) por

RULETA proporcional a su aptitud, y sobre cada una se repite la

misma búsqueda vecinal greedy. Esto concentra el esfuerzo de

búsqueda en las regiones más prometedoras (explotación).

3. Abejas Exploradoras: las fuentes que acumulan `trial >= limit`

intentos sin mejorar se abandonan y se reinician aleatoriamente

(diversificación / escape de mínimos locales).

"""

```
# Importaciones necesarias similares a las actividades anteriores
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
```

```
from typing import Callable, Protocol, Sequence
```



```
import numpy as np

from numpy.random import Generator

"""

1. Especificación del problema (función objetivo + dominio)

-----

Misma idea y contratos de las actividades pasadas

"""

@dataclass(frozen=True)

class ProblemSpec:

    """Describe un problema de minimización con dominio tipo
    caja."""

    name: str

    func: Callable[[np.ndarray], np.ndarray]

    bounds: Sequence[tuple[float, float]]

    known_optimum: float | None = None

    @property

    def n_vars(self) -> int:

        return len(self.bounds)

def _eggholder(x: np.ndarray) -> np.ndarray:

    """Función Eggholder (2D), x, y en [-512, 512].

    Óptimo global conocido: f(512, 404.2319) ~= -959.6407.

    Superficie no convexa, extremadamente multimodal."""

    xv, yv = x[:, 0], x[:, 1]
```

```
term1 = -(yv + 47) * np.sin(np.sqrt(np.abs(xv / 2.0 + (yv + 47))))

term2 = -xv * np.sin(np.sqrt(np.abs(xv - (yv + 47))))

return term1 + term2


def make_eggholder() -> ProblemSpec:

    """Construye el problema Eggholder, x, y en [-512, 512]."""

    return ProblemSpec(

        name="eggholder",

        func=_eggholder,

        bounds=[(-512.0, 512.0), (-512.0, 512.0)],

        known_optimum=-959.6407,

    )


"""

# 2. Configuración del algoritmo

_____

"""


@dataclass

class ABCConfig:

    """Hiperparámetros y opciones del ABC.

    Attributes:

        colony_size: Número de fuentes de alimento o potenciales
soluciones (SN). También el número

        de abejas empleadas (una por fuente).
```

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
ALGORITMOS METAHEURÍSTICOS
Prof, Ángel Ignacio Ángel Paredes

`onlooker_count`: Número de abejas observadoras. Si es `None`, se usa

`colony_size` (proporción 1:1 clásica).

`limit`: Número de intentos sin mejorar antes de abandonar una fuente y llamar a una abeja exploradora.

Si es `None`, se calcula como `colony_size * n_vars` (regla común

`limit = SN x D`).

`max_cycles`: Criterio de paro FUERTE (tope duro de ciclos).

`neighbor_selection`: Operador registrado en `NEIGHBOR_SELECTION_OPERATORS`

para elegir el vecino `k` en la búsqueda vecinal.

`scout_strategy`: Operador registrado en `SCOUT_STRATEGIES` para decidir

qué fuentes se abandonan cuando superan ``limit``.

`stagnation_cycles`: Paro DÉBIL: ciclos sin mejora significativa del

mejor histórico antes de detenerse (`0` = desactivado).

`stagnation_epsilon`: Mejora mínima considerada significativa.

`diversity_epsilon`: Criterio de paro OPCIONAL adicional: si la

diversidad euclidiana normalizada de la población cae por

debajo de este umbral, se considera colapso de diversidad.

`0.0` = desactivado (default).

`success_tolerance`: Tolerancia `|f_encontrado - f_óptimo|` para éxito.

Dado el rango amplio y la rugosidad de Eggholder, `1.0` es un

default razonable (ajustable).

`seed`: Semilla para reproducibilidad.

"""

```
colony_size: int = 40

onlooker_count: int | None = None

limit: int | None = None

max_cycles: int = 500

neighbor_selection: str = "random"

scout_strategy: str = "single_worst"

stagnation_cycles: int = 60

stagnation_epsilon: float = 1e-6

diversity_epsilon: float = 0.0

success_tolerance: float = 1.0

seed: int | None = None
```

"""

3. Operadores: selección de vecino (Protocol + registry)

"""

```
# Protocolo de la selección de vecindad

class NeighborSelectionOperator(Protocol):

    def __call__(

        self, population: np.ndarray, index_i: int, rng: Generator,
        config: ABCConfig

    ) -> int:

        """Devuelve el índice k (!= index_i, es decir, distinto de
        él mismo) del vecino a usar en la

            búsqueda vecinal  $v_{ij} = x_{ij} + \phi_{ij}(x_{ij} - x_{kj})$ ."""

        ...
```

```
def select_neighbor_random(
    population: np.ndarray, index_i: int, rng: Generator, config:
ABCCConfig
) -> int:
    """ABC clásico: vecino aleatorio distinto de index_i (mayor
    exploración,
    ya que la dirección de búsqueda no está sesgada hacia
    soluciones
    similares)."""
    n = len(population)
    if n <= 1: # Protección en caso de que la población sea igual a
1. No hay vecinos
        return index_i
    k = index_i
    while k == index_i:
        k = int(rng.integers(0, n)) # Selección aleatoria
    return k

# Selección por distancia euclidiana
def select_neighbor_nearest_euclidean(
    population: np.ndarray, index_i: int, rng: Generator, config:
ABCCConfig
) -> int:
    """Variante: vecino más cercano en distancia euclidiana
    (excluyendo la
    propia fuente). Al perturbar hacia una solución similar, el
    paso
     $(x_{ij} - x_{kj})$  tiende a ser pequeño, favoreciendo una búsqueda
    más fina
    y local (más explotación, menos exploración que la variante
    aleatoria).
```

```
El riesgo es no acabar de explorar el espacio de búsqueda

"""

diffs = population - population[index_i]

dists = np.linalg.norm(diffs, axis=1)

dists[index_i] = np.inf # excluir la propia fuente

return int(np.argmin(dists))


# Listado de estrategias de la selección de vecinos
NEIGHBOR_SELECTION_OPERATORS: dict[str, NeighborSelectionOperator]
= {

    "random": select_neighbor_random,

    "nearest_euclidean": select_neighbor_nearest_euclidean,

}


"""

4. Operadores: estrategia de exploradoras (Protocol + registry)

-----

"""

# Contrato de la estrategia de selección de las exploradoras en
cuestión de cuantas fuentes se pueden abandonar en cada ciclo si se
supera el límite

class ScoutStrategy(Protocol):

    def __call__(self, trial_counts: np.ndarray, limit: int, rng:
Generator) -> list[int]:

        """Devuelve los índices de fuentes a abandonar/reiniciar
este ciclo."""

        ...
```

```
def scout_single_worst(trial_counts: np.ndarray, limit: int, rng:
Generator) -> list[int]:

    """ABC clásico (Karaboga): como máximo UNA fuente por ciclo -
    la de

    mayor `trial` - se abandona, y solo si supera `limit`."""

    idx = int(np.argmax(trial_counts))

    if trial_counts[idx] >= limit:

        return [idx]

    return []

def scout_all_exceeding(trial_counts: np.ndarray, limit: int, rng:
Generator) -> list[int]:

    """Variante: TODAS las fuentes que superen `limit` se abandonan
    en el

    mismo ciclo (diversificación más agresiva, útil si se busca
    escapar

    más rápido de zonas de estancamiento)."""

    return [int(i) for i in np.where(trial_counts >= limit)[0]]

# Listado de estrategias de las abejas exploradoras
SCOUT_STRATEGIES: dict[str, ScoutStrategy] = {

    "single_worst": scout_single_worst,

    "all_exceeding": scout_all_exceeding,

}
```

"""

5. Métrica de diversidad poblacional (distancia euclidiana)

Esto para evitar que todas las abejas se concentren en una fuente de alimento muy cerca

"""

```
def population_diversity_euclidean(population: np.ndarray,
bounds_arr: np.ndarray) -> float:

    """Diversidad poblacional: distancia euclidiana MEDIA de las
    fuentes

    al centroide de la población, normalizada por la diagonal del
    dominio

    (para obtener un valor en una escala interpretable, ~0 =
    colapsada,

    valores mayores = más dispersa), independiente de la escala del
    problema."""

    centroid = population.mean(axis=0)

    dists = np.linalg.norm(population - centroid, axis=1)

    mean_dist = float(dists.mean())

    domain_diagonal = float(np.linalg.norm(bounds_arr[:, 1] -
    bounds_arr[:, 0]))

    return mean_dist / domain_diagonal if domain_diagonal > 0 else
0.0
```

"""

6. Criterios de paro (combinables: fuerte + débil + diversidad
opcional)

Mista idea que con las actividades anteriores + diversidad
opcional

"""

#Protocolo del criterio de Paro


```
class StoppingCriterion(Protocol):

    def should_stop(self, cycle: int, best_history: list[float],
population: np.ndarray) -> bool:

        ...


@dataclass
class MaxCyclesStopping:

    """Criterio de paro FUERTE: tope duro de ciclos."""

    max_cycles: int

    def should_stop(self, cycle: int, best_history: list[float],
population: np.ndarray) -> bool:

        return cycle >= self.max_cycles


@dataclass
class StagnationStopping:

    """Criterio de paro DÉBIL: se detiene si el mejor histórico no
mejora

    más que `epsilon` durante `patience` ciclos consecutivos."""

    patience: int

    epsilon: float

    def should_stop(self, cycle: int, best_history: list[float],
population: np.ndarray) -> bool:

        if self.patience <= 0 or len(best_history) <=
self.patience:

            return False
```

```
recent = best_history[-(self.patience + 1):]  
  
improvement = recent[0] - min(recent)  
  
return improvement < self.epsilon
```

```
@dataclass
```

```
class DiversityStopping:
```

```
    """Criterio OPCIONAL: se detiene si la diversidad euclidiana  
    normalizada
```

```
    de la población cae por debajo de `epsilon` (colapso de  
    diversidad).
```

```
    Desactivado si epsilon <= 0.
```

```
    Usamos la función creada en la sección anterior
```

```
    """
```

```
    epsilon: float
```

```
    bounds_arr: np.ndarray
```

```
    def should_stop(self, cycle: int, best_history: list[float],  
population: np.ndarray) -> bool:
```

```
        if self.epsilon <= 0:
```

```
            return False
```

```
            return population_diversity_euclidean(population,  
self.bounds_arr) < self.epsilon
```

```
@dataclass
```

```
class CombinedStopping:
```

```
    """Combina varios criterios: se detiene si CUALQUIERA se  
    cumple."""
```

```
criteria: list[StoppingCriterion]

def should_stop(self, cycle: int, best_history: list[float],
population: np.ndarray) -> bool:

    return any(c.should_stop(cycle, best_history, population)
for c in self.criteria)

def build_stopping_criterion(config: ABCCConfig, bounds_arr:
np.ndarray) -> CombinedStopping:

    return CombinedStopping(

        criteria=[

            MaxCyclesStopping(max_cycles=config.max_cycles), #
Criterio de Paro Fuerte

            StagnationStopping(patience=config.stagnation_cycles,
epsilon=config.stagnation_epsilon), # Criterio de Paro Débil

            DiversityStopping(epsilon=config.diversity_epsilon,
bounds_arr=bounds_arr), # Criterio de Paro Extra para la Diversidad
Poblacional

        ]

    )

"""

7. Resultado de una corrida

-----

Misma idea que las actividades pasadas, un contenedor de datos

"""

@dataclass
```

```
class RunResult:

    """Resultado de una corrida completa del ABC."""

    best_x: np.ndarray

    best_fitness: float

    cycles_run: int

    convergence_best: list[float] = field(default_factory=list)    #
    mejor histórico por ciclo

    diversity_history: list[float] = field(default_factory=list)    #
    diversidad euclidiana por ciclo

    stopped_reason: str = ""

CycleCallback = Callable[[int, np.ndarray, np.ndarray], None]

"""

8. Motor ABC (agnóstico de los operadores concretos)

-----

"""

class ABCAlgorithm:

    """Motor del Algoritmo de Colonia de Abejas Artificiales.
    Obtiene los

    operadores concretos (selección de vecino, estrategia de
    exploradoras)

    de los registries según `config`, por lo que un cambio es solo
    cambiar un campo de ABCConfig, sin tocar esta clase."""

    def __init__(self, problem: ProblemSpec, config: ABCConfig) ->
    None:

        self.problem = problem # Problema a Resolver
```

```
self.config = config # Configuraciones

self.low = np.array([b[0] for b in problem.bounds],
dtype=np.float64) # Los límites bajos de cada dimensión

self.high = np.array([b[1] for b in problem.bounds],
dtype=np.float64) # Los límites altos de cada dimensión

self.bounds_arr = np.array(problem.bounds,
dtype=np.float64) # Arreglo de Límites

self.n_vars = problem.n_vars # Número de variables del
problema


self.colony_size = config.colony_size # Tamaño de la
colonia SN (Cantidad de Abejas Empleadas)

self.onlooker_count = config.onlooker_count if
config.onlooker_count is not None else config.colony_size #
Cantidad de abejas observadoras según la estrategia seleccionada

self.limit = config.limit if config.limit is not None else
config.colony_size * self.n_vars # Límite de estancamiento de una
fuente de alimento


self.neighbor_fn =
self._lookup(NEIGHBOR_SELECTION_OPERATORS,
config.neighbor_selection, "selección de vecino") # Función para
seleccionar al vecino

self.scout_fn = self._lookup(SCOUT_STRATEGIES,
config.scout_strategy, "estrategia de exploradoras") # Función para
las abejas exploradoras


# Función auxiliar para validar que las estrategias existan en
sus listas

@staticmethod
def _lookup(registry: dict, key: str, label: str):

    try:

        return registry[key]

    except KeyError as e:
```

```
        raise ValueError(f"Operador de {label} '{key}' no
registrado. Disponibles: {list(registry)}") from e

# Generación aleatoria de las fuentes de alimento

def _random_population(self, size: int, rng: Generator) ->
np.ndarray:

    return self.low + rng.random((size, self.n_vars)) *
(self.high - self.low)

# Evaluación del fitness de la población de fuentes

def _evaluate(self, population: np.ndarray) -> np.ndarray:

    return self.problem.func(population)

def _neighbor_search_candidate(self, population: np.ndarray,
index_i: int, rng: Generator) -> np.ndarray:

    """Genera una solución candidata a partir de la fuente
`index_i`

    mediante la fórmula vecinal  $v_{ij} = x_{ij} + \phi_{ij}(x_{ij} -$ 
 $x_{kj})$ ,

    modificando una única dimensión  $j$  elegida al azar (esquema
clásico

    de ABC, que perturba un componente a la vez).

    Es la exploración de los bordes de una fuente de alimento
que se vio en clase

    """

    k = self.neighbor_fn(population, index_i, rng, self.config)
# Selección del vecino

    j = int(rng.integers(0, self.n_vars)) # Selección de una
dimensión al azar

    phi = rng.uniform(-1.0, 1.0) # Número aleatorio phi
pertence a [-1,1]
```

```
        candidate = population[index_i].copy() # Fuente de alimento
original

        candidate[j] = population[index_i, j] + phi *
(population[index_i, j] - population[k, j]) # Modificación de la
dimensión en base a la fórmula vecindad

        return np.clip(candidate, self.low, self.high) # En caso de
que se salga del espacio de búsqueda, se corta a los bordes


def _greedy_update(
    self,
    population: np.ndarray,
    fitness: np.ndarray,
    trial_counts: np.ndarray,
    index_i: int,
    rng: Generator,
) -> None:

    """Genera un candidato vecino a `index_i` y lo adopta
(selección

    greedy) si mejora la fuente actual; si no, incrementa su
contador

    de intentos fallidos (`trial`)."""

    candidate = self._neighbor_search_candidate(population,
index_i, rng) # Encontrar una posible nueva fuente de alimento

    f_candidate = float(self.problem.func(candidate.reshape(1,
-1))[0]) # Evaluación del fitness de la nueva fuente


    if f_candidate < fitness[index_i]: # Si la fuente candidata
es mejor (Menor Fitness)

        population[index_i] = candidate # Actualizamos la
fuente

        fitness[index_i] = f_candidate # Actualizamos el
fitness de la fuente
```

```
        trial_counts[index_i] = 0 # Reiniciamos el contador de
estancamiento

        else: # Si no fue mejor

            trial_counts[index_i] += 1 # Aumentamos el contador de
estancamiento


def _employed_phase(

    self, population: np.ndarray, fitness: np.ndarray,
    trial_counts: np.ndarray, rng: Generator

) -> None:

    """Fase de abejas Empleadas: cada fuente intenta una
búsqueda

    vecinal (una abeja empleada por fuente)."""

    for i in range(self.colony_size): # Cada abeja en la
colonia

        self._greedy_update(population, fitness, trial_counts,
i, rng) # Intenta buscar una mejor fuente de alimento, puede
conseguirlo o aumentar su contador de estancamiento


    # Selección de probabilidades que usarán las abejas
observadoras por método de ruleta

    def _selection_probabilities(self, fitness: np.ndarray) ->
np.ndarray:

        """Transforma f(x) (a minimizar) en probabilidades de
selección

        para la ruleta de observadoras, con la fórmula estándar de
ABC:

            fit(x) = 1 / (1 + f(x))      si f(x) >= 0

            fit(x) = 1 + |f(x)|          si f(x) < 0

        (garantiza aptitud positiva y monótona decreciente en f,
incluso

        con valores de f negativos como en Eggholder)."""
```



```
fit = np.where(fitness >= 0, 1.0 / (1.0 + fitness), 1.0 +
np.abs(fitness))

return fit / fit.sum()

def _onlooker_phase(
    self, population: np.ndarray, fitness: np.ndarray,
    trial_counts: np.ndarray, rng: Generator
) -> None:
    """Fase de abejas Observadoras: se eligen `onlooker_count`
    fuentes
    (con reemplazo) por ruleta proporcional a su aptitud, y
    sobre cada
    una se repite la búsqueda vecinal greedy (concentra el
    esfuerzo de
    búsqueda en las zonas más prometedoras)."""
    probs = self._selection_probabilities(fitness) # Cálculo de
    probabilidades para la ruleta
    chosen = rng.choice(self.colony_size,
    size=self.onlooker_count, replace=True, p=probs) # Selección
    aleatoria de onlooker_count fuentes pudiéndose repetir
    for i in chosen:
        self._greedy_update(population, fitness, trial_counts,
        int(i), rng) # Cada observadora intenta buscar una fuente de
        alimento mejor dada en referencia a una fuente de alimento vecino
        # Comportamiento idéntico a la empleadas

def _scout_phase(
    self, population: np.ndarray, fitness: np.ndarray,
    trial_counts: np.ndarray, rng: Generator
) -> None:
    """Fase de abejas Exploradoras: las fuentes que superan
    `limit`
    intentos sin mejorar (según la estrategia configurada) se
```

```
abandonan y se reinician con una posición aleatoria
uniforme."""

    to_reset = self.scout_fn(trial_counts, self.limit, rng) #
    Se seleccionan las fuentes que serán abandonas según la estrategia

    for idx in to_reset: # Por cada una

        population[idx] = self.low + rng.random(self.n_vars) *
        (self.high - self.low) # Inicializamos la fuente en un punto
        aleatorio en el espacio de búsqueda

        fitness[idx] =
        float(self.problem.func(population[idx].reshape(1, -1))[0]) #
        Calculamos el fitness de la nueva fuente

        trial_counts[idx] = 0 # Reiniciamos el contador de
        estancamiento

    def run(self, rng: Generator | None = None, callback:
    CycleCallback | None = None) -> RunResult:

        """Ejecuta el ABC completo: cada ciclo aplica, en orden, la
        fase de

        empleadas, observadoras y exploradoras."""

        rng = rng if rng is not None else
        np.random.default_rng(self.config.seed) # Motor de números
        aleatorios

        stopping = build_stopping_criterion(self.config,
        self.bounds_arr) # Creación de los criterios de paro según las
        configuraciones y atributos

        population = self._random_population(self.colony_size, rng)
        # Generación aleatoria de las fuentes de alimento en el espacio de
        búsqueda

        fitness = self._evaluate(population) # Cálculo del fitness
        de las fuentes de alimento

        trial_counts = np.zeros(self.colony_size, dtype=np.int64) #
        Inicialización en 0's de los contadores de estancamiento

        # Estadística
```

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
ALGORITMOS METAHEURÍSTICOS
Prof, Ángel Ignacio Ángel Paredes

```
best_idx = int(np.argmin(fitness)) # Mejor fitness (Menor
valor)

best_x = population[best_idx].copy() # Mejor X (X para el
menor Y alcanzada)

best_fitness = float(fitness[best_idx]) # Mejor fitness
alcanzado (Menor vaqlor)


# Listas de convergencia y diversidad del problema

convergence_best: list[float] = []

diversity_history: list[float] = []


cycle = 0 # Contador de iteraciones

stopped_reason = "max_cycles" # Razón de paro (por default
será número de ciclos; será modificada si otro criterio de paro es
el que finalmente actua)


while True: # Ciclo Principal

    current_best_idx = int(np.argmin(fitness)) # Mejor
índice actual

    if fitness[current_best_idx] < best_fitness: #
Actualización del fitness y el mejor X dado el mejor índice (Menor
valor)

        best_fitness = float(fitness[current_best_idx])

        best_x = population[current_best_idx].copy()


    convergence_best.append(best_fitness) # Índice de
convergencia


diversity_history.append(population_diversity_euclidean(population,
self.bounds_arr)) # Historia de la evolución de la diversidad
```

```
        if callback is not None: # Función auxiliar si hay más
información o procesos relevantes a medio ciclo. Lo usaremos para
capturar 3 momentos de la población en una run
```

```
        callback(cycle, population, fitness)
```

```
        if stopping.should_stop(cycle, convergence_best,
population): # Evaluación de los criterios de paro
```

```
            if cycle >= self.config.max_cycles: # Paro por
máximo de ciclos (Paro Fuerte)
```

```
                stopped_reason = "max_cycles"
```

```
            elif self.config.diversity_epsilon > 0 and
diversity_history[-1] < self.config.diversity_epsilon: # Paro por
colapso de diversidad (Paro extra)
```

```
                stopped_reason = "diversity_collapse"
```

```
            else:
```

```
                stopped_reason = "stagnation" # Paro débil
```

```
            break
```

```
        # Fases principales del ABD
```

```
        self._employed_phase(population, fitness, trial_counts,
rng) # Fase de las abejas empleadas (Evaluación y búsqueda de
mejores fuentes)
```

```
        self._onlooker_phase(population, fitness, trial_counts,
rng) # Fase de las abejas observadoras (Selección de las mejores
fuentes y búsqueda de mejores fuentes)
```

```
        self._scout_phase(population, fitness, trial_counts,
rng) # Fase de las abejas exploradoras (Abandonar fuentes
estancadas y aleatorizar nuevas)
```

```
        cycle += 1
```

```
        #Retorno de resultados y estadística
```

```
        return RunResult(
```

```
            best_x=best_x,
```

```
            best_fitness=best_fitness,
```

```
        cycles_run=cycle,

        convergence_best=convergence_best,

        diversity_history=diversity_history,

        stopped_reason=stopped_reason,

    )

"""

9. Utilidad de éxito

-----

"""

def is_success(problem: ProblemSpec, best_fitness: float,
               tolerance: float) -> bool | None:

    """Determina si una corrida fue "exitosa" comparando contra el
    óptimo

    analítico conocido. None si el problema no tiene óptimo
    conocido."""

    if problem.known_optimum is None:

        return None

    return abs(best_fitness - problem.known_optimum) < tolerance
```

main.py

"""

Autor: Mariscal Rodríguez Omar Jesús

Materia: Algoritmos Metaheurísticos

Profesor: Paredes López Ángel Ignacio

Actividad 4 - Algoritmo de Colonia de Abejas Artificiales

Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

main.py

Orquesta la Actividad 4 en dos modos:

 --mode tuning (Estrategia heredada de la actividad anterior
para la búsqueda de hiperparámetros ideales):

 Barrido de candidatos de (tamaño de colonia SN,
multiplicador de

 `limit` respecto a SN x D, máximo de ciclos), con pocas
corridas

 por combinación sobre Eggholder. Genera una tabla
comparativa para

 elegir manualmente la configuración final

 --mode final (default):

 Corre la configuración elegida con ≥ 30 corridas
independientes

 sobre Eggholder. Genera:

- Tabla estadística (mejor, peor, media, desviación estándar, tasa de éxito).
- Curva de convergencia Promedio (mejor fitness vs. ciclo, promediada sobre las corridas).
- Una corrida adicional Dedicada (separada de las 30 estadísticas, con semilla propia reproducible) que captura snapshots de la población en 3 momentos (inicial, intermedia, final) y los grafica sobre un contorno de la función Eggholder para la distribución espacial
- Gráfica de diversidad poblacional (distancia euclidiana media al centroide) vs. ciclo, como información adicional que ilustra el efecto de las abejas exploradoras.

Para la modificación:

- Cambiar selección de vecino de aleatoria a la más cercana euclidiana:

```
flag --neighbor-selection nearest_euclidean (o viceversa).
```
- Cambiar estrategia de exploradoras (una por ciclo vs. todas las que excedan `limit`):

```
flag --scout-strategy all_exceeding.
```
- Ambos son solo cambios en `ABCCConfig`; `abc_colony.ABCAlgorithm` no requiere ninguna otra modificación.

Uso:

```
python main.py --mode tuning
```

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
ALGORITMOS METAHEURÍSTICOS
Prof, Ángel Ignacio Ángel Paredes

```
python main.py --mode final --colony-size 40 --limit 80
--max-cycles 500

python main.py --mode final --runs 5    # corrida rápida de
prueba

"""

# Importaciones necesarias similares a las actividades pasadas
from __future__ import annotations

import argparse
import csv
import dataclasses
import itertools
import os
import time

from dataclasses import dataclass

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

#Importaciones del archivo de configuraciones
from abc_colony import (
    ABCCConfig,
    ABCAlgorithm,
    ProblemSpec,
    RunResult,
    is_success,
    make_eggholder,
```



```
)

# Constantes para la carpeta de salida y número de veces que se
corre el experimento por default

OUTPUT_DIR_DEFAULT = "resultados"

N_RUNS_FINAL_DEFAULT = 30

"""

Utilidades comunes

-----

"""

# Correr una sola vuelta del algoritmo

def run_single(problem: ProblemSpec, config: ABCConfig, seed: int,
               callback=None) -> RunResult:

    algo = ABCAlgorithm(problem, config)

    return algo.run(rng=np.random.default_rng(seed),
                   callback=callback)

# Correr varias veces el algoritmo mientras se recogen los
resultados

def run_many(problem: ProblemSpec, config: ABCConfig, n_runs: int,
             base_seed: int) -> list[RunResult]:

    rng_seeder = np.random.default_rng(base_seed)

    results = []

    for _ in range(n_runs):

        run_seed = int(rng_seeder.integers(0, 2**31 - 1))

        cfg = dataclasses.replace(config, seed=run_seed)

        results.append(run_single(problem, cfg, run_seed))

    return results
```

```
# Cálculo de la curva promedio para las gráficas

def mean_curve(runs: list[RunResult], attr: str) -> np.ndarray:

    curves = [getattr(r, attr) for r in runs]

    max_len = max(len(c) for c in curves)

    padded = np.array([c + [c[-1]] * (max_len - len(c)) for c in
curves])

    return padded.mean(axis=0)

"""
Modo TUNING
"""

# Contenedor de datos para los resultados del modo Tuning
@dataclass
class TuningRow:

    colony_size: int

    limit: int

    max_cycles: int

    mean_best: float

    std_best: float

    success_rate: float | None

    mean_cycles: float

def parse_int_list(text: str) -> list[int]:

    return [int(v) for v in text.split(",")]

def parse_float_list(text: str) -> list[float]:
```

```
return [float(v) for v in text.split(",")]

# Ejecución del modo Tuning

def run_tuning(args: argparse.Namespace) -> list[TuningRow]:

    problem = make_eggholder() # Creación del Contenedor de Datos
    del Problema EggHolder

    colony_sizes = parse_int_list(args.tuning_colony_sizes) #
    Número de colonias a int

    limit_multipliers =
    parse_float_list(args.tuning_limit_multipliers) # Límites que se
    van a probar

    max_cycles_list = parse_int_list(args.tuning_max_cycles) #
    Máximo de ciclos por cada combinación

    combos = list(itertools.product(colony_sizes,
    limit_multipliers, max_cycles_list)) # Todas las combinaciones que
    el modo probará

    print(f"Modo TUNING: {len(combos)} combinaciones x
    {args.tuning_runs} corridas = " # Impresiones en consola

        f"{len(combos) * args.tuning_runs} ejecuciones totales
    sobre Eggholder\n")

    rows: list[TuningRow] = []

    #Bucle del modo tuning

    for i, (sn, limit_mult, max_cycles) in enumerate(combos,
    start=1):

        limit = max(1, round(limit_mult * sn * problem.n_vars)) #
        Límite default para el estancamiento

        config = ABCCConfig( # Creación de las configuraciones de la
        run

            colony_size=sn,

            limit=limit,
```

```
max_cycles=max_cycles,

neighbor_selection=args.neighbor_selection,

scout_strategy=args.scout_strategy,

stagnation_cycles=args.stagnation_cycles,

stagnation_epsilon=args.stagnation_epsilon,

success_tolerance=args.success_tolerance,

)

    print(f"[{i}/{len(combos)}] SN={sn} limit={limit}
(x{limit_mult} SN*D) max_cycles={max_cycles} "

        f"-> {args.tuning_runs} corridas...", end="",
flush=True)

    t0 = time.time() # Medición del tiempo

    runs = run_many(problem, config, args.tuning_runs,
base_seed=args.seed) # Correr la combinación

    elapsed = time.time() - t0

    #Recolectar estadística

    best_values = np.array([r.best_fitness for r in runs])

    successes = [is_success(problem, v, args.success_tolerance)
for v in best_values]

    success_rate = float(np.mean(successes)) if all(s is not
None for s in successes) else None

    # Agregar la estadística al retorno (para la exportación de
la tabla)

    rows.append(

        TuningRow(

            colony_size=sn,

            limit=limit,

            max_cycles=max_cycles,

            mean_best=float(best_values.mean()),
```

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
ALGORITMOS METAHEURÍSTICOS
Prof, Ángel Ignacio Ángel Paredes

```
        std_best=float(best_values.std(ddof=1)) if
len(best_values) > 1 else 0.0,

        success_rate=success_rate,

        mean_cycles=float(np.mean([r.cycles_run for r in
runs])),

    )

)

print(f" media={rows[-1].mean_best:.4f} ({elapsed:.1f}s)")

return rows

#Impresión en consola de los resultados del modo Tuning

def print_tuning_table(rows: list[TuningRow]) -> None:

    header = (f"{'SN':>5} {'limit':>7} {'max_cyc':>8} {'Media
mejor':>14} "

               f"{'Desv.Std':>10} {'% Éxito':>9} {'Ciclos
media':>13}")

    print("\n" + header)

    print("-" * len(header))

    for r in rows:

        success_str = f"{r.success_rate:.0%}" if r.success_rate is
not None else "N/A"

        print(

            f"{r.colony_size:>5} {r.limit:>7} {r.max_cycles:>8}
{r.mean_best:>14.4f} "

            f"{r.std_best:>10.4f} {success_str:>9}
{r.mean_cycles:>13.1f}"

        )

# Exportación de la tabla de exploración de hiperparámetros a un
CSV
```

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
ALGORITMOS METAHEURÍSTICOS
Prof, Ángel Ignacio Ángel Paredes

```
def export_tuning_csv(rows: list[TuningRow], path: str) -> None:

    with open(path, "w", newline="", encoding="utf-8") as f:

        writer = csv.writer(f)

        writer.writerow(["SN", "limit", "max_cycles",
            "media_mejor", "desv_std", "tasa_exito", "ciclos_media"])

        for r in rows:

            writer.writerow([

                r.colony_size, r.limit, r.max_cycles,
                f"{r.mean_best:.6f}", f"{r.std_best:.6f}",
                f"{r.success_rate:.4f}" if r.success_rate is not
                None else "N/A", f"{r.mean_cycles:.2f}",

                ])

            print(f"\nTabla de sintonización exportada a: {path}")

            print("Recuerda: 'limit' pequeño -> las fuentes se abandonan
                muy pronto (posible pérdida de "

                    "soluciones buenas antes de explotarlas). 'limit' grande
                    -> las exploradoras casi no se "

                    "activan y el enjambre puede estancarse en mínimos
                    locales. Revisa la tabla para elegir "

                    "un balance y luego corre --mode final con esos
                    valores.")

            """

            Modo FINAL

            _____

            """

# Contenedor de datos estadísticos

@dataclass

class StatsRow:
```

```
problem_name: str

best: float

worst: float

mean: float

std: float

success_rate: float | None

mean_cycles: float


# Cálculos de las estadísticas dado el éxito (si se conoce el
# mínimo analítico)

def compute_stats(problem: ProblemSpec, runs: list[RunResult],
tolerance: float) -> StatsRow:

    values = np.array([r.best_fitness for r in runs])

    successes = [is_success(problem, v, tolerance) for v in values]

    success_rate = float(np.mean(successes)) if all(s is not None
for s in successes) else None

    return StatsRow(

        problem_name=problem.name,

        best=float(values.min()),

        worst=float(values.max()),

        mean=float(values.mean()),

        std=float(values.std(ddof=1)) if len(values) > 1 else 0.0,

        success_rate=success_rate,

        mean_cycles=float(np.mean([r.cycles_run for r in runs])),

    )


# Impresión de la tabla de estadísticas

def print_stats_table(row: StatsRow) -> None:
```

UNIVERSIDAD DE GUADALAJARA
CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
ALGORITMOS METAHEURÍSTICOS
Prof, Ángel Ignacio Ángel Paredes

```
header = f"{'Problema':<12} {'Mejor':>12} {'Peor':>12}
{'Media':>12} {'Desv.Std':>12} {'% Éxito':>9} {'Ciclos media':>13}"

print("\n" + header)

print("-" * len(header))

success_str = f"{row.success_rate:.0%}" if row.success_rate is
not None else "N/A"

print(

    f"{row.problem_name:<12} {row.best:>12.4f}
{row.worst:>12.4f} {row.mean:>12.4f} "

    f"{row.std:>12.4f} {success_str:>9}
{row.mean_cycles:>13.1f}"

)

# Exportación de las estadísticas a un CSV

def export_stats_csv(row: StatsRow, path: str) -> None:

    with open(path, "w", newline="", encoding="utf-8") as f:

        writer = csv.writer(f)

        writer.writerow(["problema", "mejor", "peor", "media",
"desv_std", "tasa_exito", "ciclos_media"])

        writer.writerow([

            row.problem_name, f"{row.best:.6f}",
f"{row.worst:.6f}", f"{row.mean:.6f}", f"{row.std:.6f}",

            f"{row.success_rate:.4f}" if row.success_rate is not
None else "N/A", f"{row.mean_cycles:.2f}",

        ])

    print(f"\nTabla estadística exportada a: {path}")

#Gráfica de convergencia

def plot_convergence(runs: list[RunResult], output_path: str,
config_label: str) -> None:

    fig, ax = plt.subplots(figsize=(10, 6))

    curve = mean_curve(runs, "convergence_best")
```



```
ax.plot(curve, linewidth=1.8, color="tab:blue")

ax.set_xlabel("Ciclo")

ax.set_ylabel("Mejor fitness histórico (promedio de las
corridas)")

ax.set_title(f"Curva de convergencia promedio -
Eggholder\n({config_label})")

ax.grid(True, alpha=0.3)

fig.tight_layout()

fig.savefig(output_path, dpi=150)

plt.close(fig)

print(f"Gráfica de convergencia guardada: {output_path}")


def plot_diversity(runs: list[RunResult], output_path: str,
config_label: str) -> None:

    """Gráfica adicional: diversidad poblacional (distancia
    euclidiana

    media al centroide, normalizada) vs. ciclo para ilustrar el
    efecto de las

    abejas exploradoras reinyectando diversidad."""

    fig, ax = plt.subplots(figsize=(10, 6))

    curve = mean_curve(runs, "diversity_history")

    ax.plot(curve, linewidth=1.8, color="tab:green")

    ax.set_xlabel("Ciclo")

    ax.set_ylabel("Diversidad euclidiana normalizada (promedio de
    las corridas)")

    ax.set_title(f"Diversidad poblacional vs. ciclo -
    Eggholder\n({config_label})")

    ax.grid(True, alpha=0.3)

    fig.tight_layout()

    fig.savefig(output_path, dpi=150)
```

```
plt.close(fig)

print(f"Gráfica de diversidad poblacional guardada:
{output_path}")

def plot_spatial_distribution(
    problem: ProblemSpec, snapshots: list[tuple[int, np.ndarray]],
    output_path: str, config_label: str
) -> None:
    """Grafica 3 snapshots de la población (inicial, intermedia,
    final)
    sobre un contorno de la función Eggholder, para visualizar cómo
    el
    enjambre explora el paisaje multimodal."""
    xs = np.linspace(problem.bounds[0][0], problem.bounds[0][1],
250)
    ys = np.linspace(problem.bounds[1][0], problem.bounds[1][1],
250)
    xx, yy = np.meshgrid(xs, ys)
    grid_points = np.column_stack([xx.ravel(), yy.ravel()])
    zz = problem.func(grid_points).reshape(xx.shape)

    labels = ["Generación inicial", "Generación intermedia",
"Generación final"]

    fig, axes = plt.subplots(1, 3, figsize=(19, 6), sharex=True,
sharey=True)

    contour = None

    for ax, (cycle, population), label in zip(axes, snapshots,
labels):
        contour = ax.contourf(xx, yy, zz, levels=40,
cmap="viridis", alpha=0.8)
```

```
ax.scatter(

    population[:, 0], population[:, 1],

    c="red", edgecolors="white", s=35, linewidths=0.6,
zorder=5,

)

ax.set_title(f"{label} (ciclo {cycle})")

ax.set_xlabel("x")

axes[0].set_ylabel("y")

fig.suptitle(f"Distribución espacial de la población –
Eggholder\n({config_label})")

fig.colorbar(contour, ax=axes, shrink=0.85, label=f"(x, y)")

fig.savefig(output_path, dpi=150)

plt.close(fig)

print(f"Gráfica de distribución espacial guardada:
{output_path}")

def capture_visualization_run(problem: ProblemSpec, config:
ABCCConfig, seed: int) -> list[tuple[int, np.ndarray]]:

    """Corre el ABC una vez, capturando la población en cada ciclo,
y

    devuelve exactamente 3 snapshots (inicial, intermedia, final)
para el

    la distribución espacial pedida. Esta corrida es independiente
de

    las N_RUNS estadísticas, para no cargarlas de snapshots
innecesarios."""

    history: list[tuple[int, np.ndarray]] = []

    def collector(cycle: int, population: np.ndarray, fitness:
np.ndarray) -> None:
```

```
history.append((cycle, population.copy()))

run_single(problem, config, seed=seed, callback=collector)

initial = history[0]
final = history[-1]
intermediate = history[len(history) // 2]
return [initial, intermediate, final]

# Ejecución final del algoritmo
def run_final(args: argparse.Namespace) -> None:
    problem = make_eggholder() # Creación del Spec Problem
    config = ABCConfig( # Configuraciones según lo puesto en
consola

        colony_size=args.colony_size,
        onlooker_count=args.onlooker_count,
        limit=args.limit,
        max_cycles=args.max_cycles,
        neighbor_selection=args.neighbor_selection,
        scout_strategy=args.scout_strategy,
        stagnation_cycles=args.stagnation_cycles,
        stagnation_epsilon=args.stagnation_epsilon,
        diversity_epsilon=args.diversity_epsilon,
        success_tolerance=args.success_tolerance,
    )

    effective_limit = config.limit if config.limit is not None else
config.colony_size * problem.n_vars # Cálculo de límite de
estancamiento

    config_label = (
```

```
f"SN={config.colony_size}, limit={effective_limit},
max_cycles={config.max_cycles}, "

    f"vecino={config.neighbor_selection},
exploradoras={config.scout_strategy}"

)

print("=" * 70)

print("Actividad 4 - Algoritmo de Colonia de Abejas
Artificiales (modo FINAL)")

print(f"Corridas estadísticas: {args.runs}")

print(config_label)

print("=" * 70)

print(f"\nEjecutando {args.runs} corridas estadísticas...",
end="", flush=True)

t0 = time.time()

runs = run_many(problem, config, args.runs,
base_seed=args.seed)

elapsed = time.time() - t0

print(f" listo ({elapsed:.1f}s)")

stats_row = compute_stats(problem, runs,
args.success_tolerance)

print_stats_table(stats_row)

os.makedirs(args.output_dir, exist_ok=True)

export_stats_csv(stats_row, os.path.join(args.output_dir,
"tabla_estadistica_abc.csv"))

plot_convergence(runs, os.path.join(args.output_dir,
"convergencia_eggholder.png"), config_label)

plot_diversity(runs, os.path.join(args.output_dir,
"diversidad_eggholder.png"), config_label)
```

```
# --- Corrida dedicada para distribución espacial (semilla
propia, reproducible) ---

viz_seed = args.seed + 999_999

print(f"\nCorrida dedicada para distribución espacial
(semilla={viz_seed})...", end="", flush=True)

t0 = time.time()

snapshots = capture_visualization_run(problem, config,
seed=viz_seed)

elapsed = time.time() - t0

print(f" listo ({elapsed:.1f}s)")

plot_spatial_distribution(
    problem, snapshots, os.path.join(args.output_dir,
"distribucion_espacial_eggholder.png"), config_label
)

print("\nExperimento final completo. Resultados en:",
os.path.abspath(args.output_dir))

"""

CLI para cambiar estretegias directamente en consola sin modificar
código

_____

"""

def parse_args() -> argparse.Namespace:

    parser = argparse.ArgumentParser(description="Actividad 4 -
ABC: tuning y experimento final sobre Eggholder")

    parser.add_argument("--mode", type=str, default="final",
choices=["tuning", "final"])
```

```
# Comunes

parser.add_argument("--seed", type=int, default=42)

parser.add_argument("--neighbor-selection", type=str,
default="random",

                    choices=["random", "nearest_euclidean"])

parser.add_argument("--scout-strategy", type=str,
default="single_worst",

                    choices=["single_worst", "all_exceeding"])

parser.add_argument("--stagnation-cycles", type=int,
default=60)

parser.add_argument("--stagnation-epsilon", type=float,
default=1e-6)

parser.add_argument("--diversity-epsilon", type=float,
default=0.0)

parser.add_argument("--success-tolerance", type=float,
default=1.0)

parser.add_argument("--output-dir", type=str,
default=OUTPUT_DIR_DEFAULT)

# Modo final

parser.add_argument("--runs", type=int,
default=N_RUNS_FINAL_DEFAULT)

parser.add_argument("--colony-size", type=int, default=40)

parser.add_argument("--onlooker-count", type=int, default=None,

                    help="Default: igual a --colony-size")

parser.add_argument("--limit", type=int, default=None,

                    help="Default: colony_size * n_vars (regla
SN x D)")

parser.add_argument("--max-cycles", type=int, default=500)

# Modo tuning

parser.add_argument("--tuning-runs", type=int, default=10)
```

```
parser.add_argument("--tuning-colony-sizes", type=str,
default="20,40,80")

parser.add_argument("--tuning-limit-multipliers", type=str,
default="0.5,1,2",

                        help="limit = multiplicador * SN * D")

parser.add_argument("--tuning-max-cycles", type=str,
default="200,400")

return parser.parse_args()

# Función main
def main() -> None:

    args = parse_args() # Argumentos de consola

    if args.mode == "tuning": # Para el modo tuning

        rows = run_tuning(args)

        print_tuning_table(rows)

        os.makedirs(args.output_dir, exist_ok=True)

        export_tuning_csv(rows, os.path.join(args.output_dir,
"tabla_sintonizacion_abc.csv"))

    else: # Para el modo final

        run_final(args)

if __name__ == "__main__":

    main()
```


CONCLUSIÓN

La implementación y evaluación empírica del Algoritmo de Colonia de Abejas Artificiales (ABC) sobre la función Eggholder mostró una impresionante eficacia de esta metaheurística para resolver problemas de optimización en paisajes de búsqueda altamente no convexos. El proceso de sintonización mostró que el rendimiento del enjambre depende críticamente del equilibrio matemático entre la presión de explotación local y la capacidad de exploración global. Al configurar una población de 80 individuos y un umbral de abandono (limit) de 160 iteraciones, el modelo alcanzó un balance óptimo, logrando una tasa de éxito del 100% y evadiendo sistemáticamente los múltiples mínimos locales que actúan como trampas en esta función. El seguimiento de la distribución espacial corroboró la dinámica de forrajeo teórica del algoritmo. Se observó empíricamente cómo las abejas empleadas y observadoras intensifican la convergencia dentro de las cuencas de atracción más prometedoras, mientras que la activación de las abejas exploradoras funciona como un mecanismo de escape indispensable. Al descartar por completo las fuentes de alimento agotadas e inyectar nuevas soluciones estocásticas, el algoritmo ABC aplica una política de reemplazo agresiva que previene la homogeneización de la colonia. Esta práctica confirma que la segmentación de la población en tres fases operativas dota al algoritmo de una robustez estadística superior frente al estancamiento prematuro. Más que eso, es importante que los buenos resultados que obtengamos sean, dentro de lo posible factibles de volver a obtener, no exactamente los mismos dada la naturaleza estocástica de este tipo de algoritmos, pero sí con cierta certeza de que podemos obtener resultados similares en cuanto a lo bueno que son dada la poca desviación media.

El factor de la combinación entre la exploración local con las abejas empleadas, la priorización de las mejores fuentes con las observadores y la exploración con las exploradoras hacen una combinación interesante; considerando la función Eggholder y todas sus ‘trampas’ que tiene en formas de mínimos locales por todos lados, conociendo los algoritmos que llevamos podemos intuir por qué ABC es tan bueno en este y probablemente el Genético en sus dos versiones pueda tener defectos significativos.

Primeramente, el genético binario es bastante improbable de que funcione adecuadamente, el espacio de búsqueda es bastante grande, una cantidad limitada de bits darnos que el algoritmo sea físicamente incapaz de llegar a mínimos interesantes dada la falta de área de representatividad, y crecer desmedidamente la cantidad de bits nos trae una forma exponencial de requerimiento de potencia computacional, por lo que queda descartado.

La opción de el genético continuo es el que puede competir a la hora de encontrar el mínimo global, sin embargo, en este tipo de funciones con tantos valles es importante una buena exploración del espacio de búsqueda, y las herramientas que compiten aquí son la mutación y el abandono de soluciones que ya habíamos comparado antes; el genético continuo al hacer una cruce, la solución hija conserva datos de los padres, es

decir, tiene una buena exploración local pero no tan buena o suficientemente aleatoria en el plano global.

Y el abandono de soluciones sí es completamente aleatorio, por lo que nos entrega esa fase de exploración en diversos puntos que puede ser de gran utilidad para no caer en esos mínimos locales.

Este pequeño análisis para determinar qué algoritmo es mejor es parte de nuestro trabajo conocer las ventajas y desventajas, situaciones factibles y cuáles no de cada algoritmo, no simplemente verlos como ‘cajas negras’ que encuentran el mínimo, sino saber por qué y para qué situaciones un algoritmo puede ser mejor elección que otro, así mismo que con la selección de sus hiper parámetros.